МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЕТ

по лабораторной работе № 1
«Интерфейсы ввода-вывода общего назначения (GPIO)»
Часть 2. Реализация собственного драйвера GPIO
Вариант 5. «Кодовый замок»

Выполнил(и): студент(ы) группы
ФИО: _____

ФИО: _____

Проверил: _____

Санкт-Петербург, 2026

СОДЕРЖАНИЕ

1. Цель работы . . . 3
2. Задание . . . 3
3. Логика прикладной программы . . . 3
4. Архитектура и реализация . . . 3
5. Порядок проверки . . . 4
6. Краткое руководство пользователя . . . 5
7. Выводы . . . 5
8. Исходный код драйвера . . . 5
   8.1. Файл `io_gpio.h` . . . 5
   8.2. Файл `io_gpio.c` . . . 6
   8.3. Подключение драйвера в прикладной программе . . . 7
9. Список использованных источников . . . 7

1. Цель работы

Разработать собственный программный драйвер GPIO для микроконтроллера STM32F427, подключить его к прикладной программе кодового замка и изучить управление портами через регистры микроконтроллера.

2. Задание

В части 1 лабораторной работы реализована прикладная программа кодового замка с использованием кнопки PC15 и индикаторов PD13–PD15 стенда SDK-1.1M. Во второй части выбран способ 1: разработать драйвер GPIO без использования HAL и LL и подключить его к программе.

Драйвер должен настраивать режимы контактов, возвращать текущий режим, читать и записывать один или несколько сигналов одного порта. Повторная настройка должна менять режим без кратковременной установки некорректного уровня на выходе. Прерывания не требуются. Для доступа к аппаратуре разрешено использовать тип `GPIO_TypeDef` и определения из CMSIS.

3. Логика прикладной программы

Кнопка PC15 подключена к входу с активным низким уровнем. Программа циклически считывает ее состояние. Для устранения дребезга новое состояние принимается только после того, как оно остается неизменным в течение 30 мс. Время удержания измеряется миллисекундным счетчиком HAL; нажатие короче 600 мс считается коротким, остальные — длинными.

Код замка задан массивом из восьми элементов: короткое, длинное, короткое, короткое, длинное, длинное, короткое, длинное нажатие. Верный элемент сопровождается коротким желтым импульсом, ошибочный — красным. После трех ошибок красный индикатор мигает пять секунд. После полного ввода правильного кода включается зеленый индикатор на три секунды. Если ввод не завершен за 12 секунд, желтый индикатор включается на одну секунду и ввод начинается заново. Временные интервалы отсчитываются по `HAL_GetTick()` без блокирующего ожидания.

На стенде зеленый светодиод включается высоким уровнем, красный и желтый — низким. Драйвер предоставляет логические уровни на выводах, а функция `led_write` учитывает полярность каждого индикатора.

4. Архитектура и реализация

Программа разделена на прикладную логику в `main.c` и драйвер в `io_gpio.c` с объявлениями в `io_gpio.h`. Прикладной код обращается к выводам только

через функции драйвера. Драйвер самостоятельно включает тактирование нужного порта, изменяет регистры и выполняет операции чтения и записи. Структура взаимодействия компонентов показана на Рис. 1.

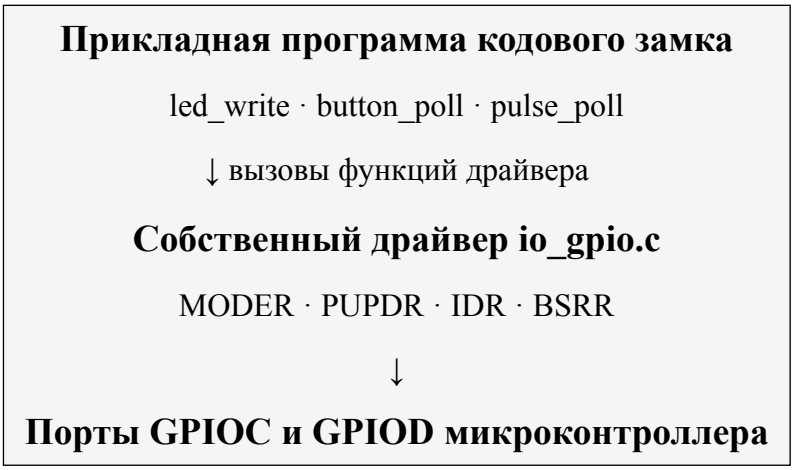


Рис. 1. Структура взаимодействия программы и драйвера GPIO

При начальной настройке кнопке задается режим входа без внутренней подтяжки, а выводам светодиодов — режим выхода. Уровни выключенных светодиодов предварительно записываются в регистр `BSRR`, затем в `MODER` включается режим выхода. При повторной инициализации в режиме выхода драйвер сначала записывает заданный начальный уровень в `BSRR` и только после этого изменяет `MODER`. Поэтому на выходе не появляется импульс от старого значения защелки `ODR`.

Для каждого выбранного вывода функция `IO_GPIO_Init` также задает поле подтяжки в `PUPDR`. `IO_GPIO_GetMode` возвращает поле режима из `MODER`. Функции `IO_GPIO_ReadPin` и `IO_GPIO_Read` читают `IDR`; `IO_GPIO_WritePin` и `IO_GPIO_Write` записывают через `BSRR`, который позволяет независимо устанавливать и сбрасывать биты без операции «прочитать-изменить-записать» регистра `ODR`.

Конфигурация выводов PC15 и PD13–PD15 удалена из `lab1.ioc`. Таким образом, после повторной генерации проекта CubeMX не создает для них HAL-настройку GPIO. В прикладном файле отсутствуют вызовы `HAL_GPIO_Init`, `HAL_GPIO_ReadPin` и `HAL_GPIO_WritePin`. HAL используется только для инициализации микроконтроллера, настройки тактирования и получения системного времени.

5. Порядок проверки

Для проверки на стенде следует загрузить проект в микроконтроллер и убедиться в следующей последовательности:

1. После запуска все три светодиода выключены.
2. Каждое устойчивое нажатие кнопки учитывается один раз; короткое и длинное нажатия различаются.
3. Верный элемент последовательности вызывает одиночный желтый импульс, неверный — одиночный красный.

4. После ввода всех восьми элементов правильного кода зеленый светодиод горит три секунды.
5. После трех неверных элементов красный светодиод мигает пять секунд.
6. Незавершенный ввод сбрасывается по таймауту с длинным желтым импульсом.

Физическая проверка на стенде должна быть выполнена перед демонстрацией преподавателю. В отчете приведен проверочный сценарий; результат измерения на стенде здесь не заявляется.

6. Краткое руководство пользователя

Для ввода кода нажимать боковую кнопку стенда. Короткое нажатие — менее 600 мс, длинное — 600 мс или больше. После каждого принятого элемента дождаться желтого импульса. Красный импульс означает неверный элемент и начало новой попытки. При совпадении всей последовательности загорится зеленый индикатор; через три секунды замок снова перейдет в режим ввода. После трех ошибок необходимо дождаться окончания блокировки.

7. Выводы

Разработан собственный драйвер GPIO для STM32F427 и подключен к программе кодового замка. Для настройки используются регистры `MODER` и `PUPDR`, для чтения — `IDR`, для изменения выходов — `BSRR`. Драйвер содержит отдельные функции для одного вывода и маски выводов. Предварительная запись уровня в `BSRR` обеспечивает корректную повторную настройку выходов без кратковременного переключения на неправильный уровень. Прикладная логика продолжает использовать опрос и неблокирующий учет времени.

8. Исходный код драйвера

Ниже приведен исходный код разработанного драйвера. Файлы HAL, CMSIS и автоматически сгенерированные файлы проекта в отчет не включены.

8.1. Файл `io_gpio.h`

```
#ifndef IO_GPIO_H
#define IO_GPIO_H

#include "stm32f4xx.h"

typedef enum
{
  IO_GPIO_INPUT = 0,
  IO_GPIO_OUTPUT = 1,
  IO_GPIO_ALTERNATE = 2,
  IO_GPIO_ANALOG = 3
} IoGpioMode;
```

```
typedef enum
{
  IO_GPIO_NO_PULL = 0,
  IO_GPIO_PULL_UP = 1,
  IO_GPIO_PULL_DOWN = 2
} IoGpioPull;

/* initial_high задает начальные уровни выходов для выбранных выводов. */
void IO_GPIO_Init(GPIO_TypeDef *port, uint16_t pins, IoGpioMode mode,
                  IoGpioPull pull, uint16_t initial_high);
IoGpioMode IO_GPIO_GetMode(GPIO_TypeDef *port, uint8_t pin_number);
uint8_t IO_GPIO_ReadPin(GPIO_TypeDef *port, uint8_t pin_number);
uint16_t IO_GPIO_Read(GPIO_TypeDef *port, uint16_t pins);
void IO_GPIO_WritePin(GPIO_TypeDef *port, uint8_t pin_number, uint8_t high);
void IO_GPIO_Write(GPIO_TypeDef *port, uint16_t pins, uint16_t high_pins);

#endif
```

8.2. Файл `io_gpio.c`

```
#include "io_gpio.h"

static void enable_port_clock(GPIO_TypeDef *port)
{
  if (port == GPIOA) RCC->AHB1ENR |= 1U << 0;
  else if (port == GPIOB) RCC->AHB1ENR |= 1U << 1;
  else if (port == GPIOC) RCC->AHB1ENR |= 1U << 2;
  else if (port == GPIOD) RCC->AHB1ENR |= 1U << 3;
  else if (port == GPIOE) RCC->AHB1ENR |= 1U << 4;
  else if (port == GPIOF) RCC->AHB1ENR |= 1U << 5;
  else if (port == GPIOG) RCC->AHB1ENR |= 1U << 6;
  else if (port == GPIOH) RCC->AHB1ENR |= 1U << 7;
  else if (port == GPIOI) RCC->AHB1ENR |= 1U << 8;

  (void)RCC->AHB1ENR;
}

void IO_GPIO_Init(GPIO_TypeDef *port, uint16_t pins, IoGpioMode mode,
                  IoGpioPull pull, uint16_t initial_high)
{
  uint8_t pin;
  uint32_t shift;

  enable_port_clock(port);

  if (mode == IO_GPIO_OUTPUT)
  {
    IO_GPIO_Write(port, pins, initial_high);
  }

  for (pin = 0; pin < 16; pin++)
  {
    if ((pins & (1U << pin)) != 0)
    {
      shift = (uint32_t)pin * 2U;
      port->PUPDR = (port->PUPDR & ~(3U << shift)) |
                    ((uint32_t)pull << shift);
      port->MODER = (port->MODER & ~(3U << shift)) |
                    ((uint32_t)mode << shift);
    }
  }
}

IoGpioMode IO_GPIO_GetMode(GPIO_TypeDef *port, uint8_t pin_number)
{
  return (IoGpioMode)((port->MODER >> ((uint32_t)pin_number * 2U)) & 3U);
}

uint8_t IO_GPIO_ReadPin(GPIO_TypeDef *port, uint8_t pin_number)
{
  return (uint8_t)((port->IDR >> pin_number) & 1U);
}
```

```
uint16_t IO_GPIO_Read(GPIO_TypeDef *port, uint16_t pins)
{
  return (uint16_t)(port->IDR & pins);
}

void IO_GPIO_WritePin(GPIO_TypeDef *port, uint8_t pin_number, uint8_t high)
{
  uint32_t pin = 1U << pin_number;
  port->BSRR = high ? pin : (pin << 16);
}

void IO_GPIO_Write(GPIO_TypeDef *port, uint16_t pins, uint16_t high_pins)
{
  uint32_t low_pins = (uint32_t)pins & ~(uint32_t)high_pins;
  port->BSRR = ((uint32_t)high_pins & pins) | (low_pins << 16);
}
```

8.3. Подключение драйвера в прикладной программе

```
IO_GPIO_Init(GPIOC, GPIO_PIN_15, IO_GPIO_INPUT, IO_GPIO_NO_PULL, 0);
IO_GPIO_Init(GPIOD, GREEN_PIN | RED_PIN | YELLOW_PIN,
             IO_GPIO_OUTPUT, IO_GPIO_NO_PULL, RED_PIN | YELLOW_PIN);

uint8_t pressed_level = IO_GPIO_ReadPin(GPIOC, 15);
IO_GPIO_Write(GPIOD, pin, high ? pin : 0);
```

9. Список использованных источников

1. Проектирование вычислительных систем. Учебное пособие и задания к лабораторным работам. Лабораторная работа 1 «Интерфейсы ввода-вывода общего назначения (GPIO)», раздел 5.1.
2. Порядок выполнения и сдачи лабораторных работ по дисциплине «Проектирование вычислительных систем». Редакция 2026 г.
3. STMicroelectronics. STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs: Reference manual RM0090. Раздел «General-purpose I/Os (GPIO)».
4. ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам. Требования к оформлению применены в соответствии с методическими указаниями к лабораторной работе.